Data Visualisation in R

Devin Nanayakkara

April 19, 2026

Week 1

1 Basic Calculations

Write R code to calculate the following:

- $45 + 30$
- $120 - 25$
- $15 * 6$
- $100/5$

Store each result in a variable called:

- $result1$
- $result2$
- $result3$
- $result4$

Print all results.

2 Using Variables

Create the following variables:

- $name \rightarrow$ your first name
- $age \rightarrow$ your age
- $city \rightarrow$ the city you live in

Print all three variables.

3 Updating Variables

Create a variable called:

```
score <- 75
```

- Increase the score by 10
- Print the updated score

4 Data Types

Use the *class*() function to identify the data type of the following:

```
class(45)
class("R Programming")
class(TRUE)
class(NA)
```

Write down the output.

5 Data Types

Create a vector called *numbers* containing:

```
4, 8, 15, 16, 23, 42
```

Then:

- Print the vector
- Find the length of the vector
- Find the sum of the numbers

Write down the output.

6 Access Vector Elements

Create the following vector:

```
fruits <- c("Apple", "Banana", "Orange", "Mango", "Grapes")
```

Retrieve:

- The first fruit
- The third fruit
- The last fruit

7 Sorting Values

Create the following vector:

```
scores <- c(85, 72, 90, 66, 78)
```

- Sort the scores in ascending order
- Find the maximum score
- Find the minimum score

8 If Statement

Create a variable:

```
temperature <- 30
```

Write an **if statement**:

- If temperature is greater than 25, print "*It is a hot day*"
- Otherwise print "*Weather is pleasant*"

9 Logical Operators

Create the following variables:

```
age <- 20
is_student <- TRUE
```

Write an **if statement** that checks: If $age >= 18$ AND $is_student == TRUE$

- If both are true print:, print "*You are an eligible student*"
- Otherwise print "*You are not eligible*"

10 Combining Text

Create the following variables:

```
name <- "Alex"
age <- 22
```

Create a new variable called *message* that prints: "*Alex is 22 years old*"

Then print the message. Hint: Use the *paste*() function.

Week 2

1 Mean (Basic)

Create a vector:

```
scores <- c(70, 85, 90, 60, 75)
```

Tasks:

- Calculate the mean
- Print the result with a message

2 Mean (Real Context)

Create a vector of daily temperatures:

```
temps <- c(22, 25, 27, 23, 26, 24, 28)
```

Tasks:

- Calculate the average temperature
- Print: *The average temperature is X*

3 Median (Odd Dataset)

Create:

```
data <- c(5, 2, 9, 1, 7)
```

Tasks:

- Sort the dataset
- Calculate the median
- Print the result

4 Median (Even Dataset)

Create:

```
data <- c(10, 4, 6, 8)
```

Tasks:

- Sort the data
- Calculate the median using:
  - Built-in function
  - Manual approach (average of middle values)

5 Mean vs Median Comparison

Create:

```
data <- c(10, 12, 14, 16, 100)
```

Tasks:

- Calculate the mean
- Calculate the median
- Compare results
- Which measure is affected by the outlier?

6 Mode (Frequency)

Create:

```
data <- c(2, 4, 4, 6, 6, 6, 8)
```

Tasks:

- Find frequency of each value using *table*()
- Identify the mode manually

7 Mode using Package

Install and load the package:

```
install.packages("DescTools")
library(DescTools)
```

Create:

```
data <- c(1, 2, 2, 3, 3)
```

Tasks:

- Use *Mode*() to find the mode(s)

8 Variance (Using Function)

Create:

```
data <- c(5, 10, 15, 20, 25)
```

Tasks:

- Calculate variance using *var*()
- Print the result

9 Variance (Manual Calculation)

Create:

```
data <- c(5, 10, 15, 20, 25)
```

Tasks:

- Calculate mean
- Subtract mean from each value
- Square the differences
- Find the average of squared values

10 Full Statistical Summary

Create:

```
data <- c(12, 15, 14, 10, 18, 20, 15)
```

Tasks:

- Mean
- Median
- Mode (manual or package)
- Variance

11 Bonus Challenge

Create two datasets:

```
data1 <- c(-2, -1, 0, 1, 2)
data2 <- c(-200, -100, 0, 100, 200)
```

Tasks:

- Compare mean and median
- Compare variance
- Explain: Why are the means the same but variances different?

Week 3

1 Basic Standard Deviation

Create a numeric vector and calculate:

- Mean
- Standard deviation

Then print both values.

2 Interpreting Spread

Using your dataset from Exercise 1:

- Compare $sd()$ with $mean()$
- Write an if statement:
  - If SD $<$ mean $\longrightarrow$ *Data is close together*
  - Else $\longrightarrow$ *Data is spread out*

3 Manual vs Built-in SD

Create a dataset and:

- Manually calculate variance
- Take square root
- Compare with $sd()$

Are the values equal?

4 Sorting and Median (Q2)

Create an unsorted dataset:

- Sort it using $sort()$
- Find median using:
  - $median()$
  - manual indexing

Do both methods match?

5 Quartiles (Manual Method)

Using a dataset, calculate:

- Q1 (lower half median)
- Q2 (median)
- Q3 (upper half median)

Follow the step-by-step slicing method from slides (weekly content)

6 Quartiles Using R

Using the same dataset, use *quantile*() to compute:

- Q1 (0.25)
- Q2 (0.50)
- Q3 (0.75)

Compare with Exercise 5 results.

7 Quartiles Using R

Using a dataset, calculate:

- 10th percentile
- 25th percentile
- 90th percentile

Use:

```
quantile(data, c(0.1, 0.25, 0.9))
```

8 Interquartile Range (IQR)

Using a dataset, calculate:

- IQR manually: $IQR = Q3 - Q1$
- IQR using $IQR()$

Compare both results.

9 Detecting Outliers

Using IQR, calculate:

- Lower bound $= Q1 - 1.5 \times IQR$
- Upper bound $= Q3 + 1.5 \times IQR$

Identify outliers in the dataset and print them.

10 Full Data Analysis Challenge

Create a dataset (e.g., exam scores or sales data) and:

- Calculate: Mean, SD, Q1, Q2, Q3, IQR
- Interpret:
  - Is data spread wide or narrow?
  - Any outliers?
  - Is distribution skewed? (based on quartiles/median)
- Use *boxplot*() and *hist*() to visualise

Week 4/5

1 Creating and Inspecting Vectors

Create a vector called $student_marks$ with the values: $65, 70, 85, 90, 75$

- Find the length of the vector
- Identify the data type

2 Vector Indexing

Using $student_marks$:

- Extract the 4th value
- Change the 2nd value to 80
- Print the updated vector

3 Logical Filtering

Create a vector $scores$ from 1 to 20:

- Extract only scores greater than 10
- Extract only even numbers
- Extract values between 5 and 15

4 Vector Operations

Create two vectors:

```
v1 <- c(2,4,6,8)
v2 <- c(1,3,5,7)
```

- Add, subtract, and multiply them element-wise
- Divide $v1$ by $v2$

5 Matrix Creation

Create a 3×3 matrix called $matrixA$ with values from 1 to 9:

- Print the matrix
- Check number of rows and columns

6 Matrix Indexing

Using $matrixA$:

- Extract the element in row 2, column 3
- Extract the entire first row
- Extract the entire second column

7 Matrix Operations

Create another 3×3 matrix $matrixB$ with values 9 to 1:

- Add $matrixA$ and $matrixB$
- Multiply them element-wise
- Perform matrix multiplication

8 Matrix Functions

Using $matrixA$:

- Calculate row sums
- Calculate column means
- Find standard deviation of each column

9 Working with Lists

Create a list $student_info$ containing:

- Name
- Age
- Height
- A vector of 3 favourite subjects

Tasks:

- Access the age
- Update height
- Add a new element called *hobbies*

10 Data Frame Practice

Create a data frame *employees* with:

- Name: 3 names
- Salary: numeric values
- FullTime: TRUE/FALSE

Tasks:

- Print the *Salary* column
- Add a new column *Bonus* (10% of salary)
- Filter employees with $salary > 5000$

11 Optional Challenge

Create a data frame of 5 students with 3 exam scores:

- Calculate the **average score per student**
- Add a column $Pass$ where average ≥ 50

Week 6

1 Even or Odd

Write an R program that:

- Takes a number *num*
- Prints *Even* if divisible by 2, otherwise *Odd*
- Print the updated vector

2 Temperature Check

Create a variable *temp* and:

- If $temp > 30 \longrightarrow$ print *Hot*
- If *temp* between 20 and 30 $\longrightarrow$ print *Warm*
- Otherwise $\longrightarrow$ print *Cold*

3 Grade with Feedback

Extend the grading logic:

- $\geq 85 \longrightarrow$ *High Distinction*
- $\geq 75 \longrightarrow$ *Distinction*
- $\geq 65 \longrightarrow$ *Credit*
- $\geq 50 \longrightarrow$ *Pass*
- Else $\longrightarrow$ *Fail*

Also print *Well done!* for HD or D.

4 Nested Condition (Login System)

Create variables:

```
username <- "admin"
password <- "1234"
```

- If username is correct:
  - Check password:
    - Correct $\longrightarrow$ *Access granted*
    - Wrong $\longrightarrow$ *Incorrect password*
- Else $\longrightarrow$ *User not found*

5 Sum of Even Numbers

- Create a vector:

```
numbers <- 1:20
```

- Use a for loop to calculate the sum of only even numbers

6 Multiplication Table

Write a loop that prints:

```
5 x 1 = 5
5 x 2 = 10
...
5 x 10 = 50
```

7 Countdown with while loop

- Start from 10
- Print numbers down to 1
- Print *Blast off!* at the end

8 Running Total Until Limit

- Keep adding random numbers $(1 - 10)$ using *sample*()
- Stop when total exceeds 50
- Print the final total

Directly builds on the coin flip logic in weekly content slides

9 Custom Function – Area of Circle

Write a function:

```
area_circle <- function(radius)
```

- Returns area: πr^2
- Test with different radius values

10 Apply Function Practice

Given:

```
marks <- c(45, 67, 82, 90, 55)
```

- Create a function:

```
pass_fail <- function(x)
```

- Returns *Pass* if ≥ 50, else *Fail*

Use *sapply*() to apply this function to all marks

11 Optional Challenge - Matrix + apply()

- Create a 3x3 matrix
- Use *apply*() to:
  - Find row sums
  - Find column means

Week 7

1 Create a Data Frame

Create a data frame called *employees* with the following columns:

- *name*: "Alice", "Bob", "Charlie"
- *age*: 25, 30, 35
- *salary*: 50000, 60000, 70000

Display the data frame.

2 Inspect a Data Frame

Using the built-in *mtcars* dataset:

- Display the first 6 rows
- Display the first 10 rows
- Generate summary statistics

3 Load and Save CSV

- Load a CSV file called "*sales.csv*" into a data frame called *sales_df*
- Display the first 5 rows
- Save it as "*sales_copy.csv*"

4 Select Columns (dplyr)

Using *mtcars*:

- Select only the columns: *mpg*, *hp*, and *cyl*

5 Filter Rows (Single Condition)

Using *mtcars*:

- Filter cars with *mpg* greater than 25
- Store in *high_mpg*

6 Filter Rows (Multiple Conditions)

Using *mtcars*:

- Filter cars with:
  - $cyl == 4$
  - $hp > 90$
- Store in $powerful_small_cars$

7 Arrange Data

Using $mtcars$:

- Sort cars by mpg in ascending order
- Then sort cars by hp in descending order

8 Piping (%>%)

Using $mtcars$, write a single pipeline to:

- Filter cars with $mpg > 20$
- Select only mpg, hp, and cyl

9 Combined Operations

Using $mtcars$:

- Filter cars with $cyl == 6$
- Arrange them by mpg descending
- Show the first 5 rows

Use piping (%>%)

10 Mini Case Study

Using $mtcars$:

- Filter cars with $hp > 100$
- From those, select mpg, hp, cyl
- Sort by mpg descending
- Calculate the **average mpg**

Store final result as $performance_cars$

Week 8

Download dataset file from https://github.com/dvn14/Data-Viz-in-R

1 Diagnosing Data

Create a data frame with at least 3 columns and:

- Display the first 6 rows
- Show summary statistics
- Print column names

Functions to use: *head*(), *summary*(), *colnames*()

2 Understanding Structure

Load a dataset (or create one) and:

- Check the structure of the dataset
- Count number of rows and columns

Functions to use: *str*(), *nrow*(), *ncol*()

3 Combining Multiple Files

Assume you have multiple CSV files named: *data_1.csv*, *data_2.csv*, *data_3.csv*. Write code to:

- List all files
- Read them into R
- Combine them into one dataset

Functions: *list.files*(), *lapply*(), *read.csv*(), *bind_rows*()

4 Reshaping Data (Wide $\longrightarrow$ Long)

Given a dataset:

```
df <- data.frame(
    ID = c(1,2,3),
    Math = c(80, 90, 85),
    English = c(70, 88, 92)
)
```

Convert it into tidy format with:

- Column: *Subject*
- Column: *Score*

Use: *gather*() or *pivot_longer*()

5 Handling Missing Values

Create a dataset with missing values:

- Count total missing values
- Remove rows with missing values
- Replace missing values with a constant (e.g., 0)

Functions: *is.na*(), *complete.cases*()

6 Detecting and Removing Outliers

Given a numeric column:

- Identify outliers using boxplot logic
- Remove those outliers from the dataset

Hint:

```
boxplot(x, plot = FALSE)$out
```

7 Removing Duplicates

Create a dataset with duplicate rows:

- Identify duplicate rows
- Remove duplicates

Functions: *duplicated*(), *distinct*() or *unique*()

8 Splitting Columns (By Character)

Given a column like:

```
"Premium_AU"
"Basic_US"
```

Split into:

- *Plan*
- *Country*

Use: *separate*()

9 Splitting Columns (By Index)

Given:

```
birthday = "12051998"
```

Extract:

- *Month*
- *Day*
- *Year*

Use: *substr*()

10 String Cleaning & Type Conversion

Given:

```
price = "$120"
```

Extract:

- Remove $ symbol
- Convert to numeric
- Verify data type

Functions: *gsub*(), *as.numeric*(), *str*()

Week 9

Download dataset file from https://github.com/dvn14/Data-Viz-in-R

1 Basic Aggregate – Total

Given a dataset *sales* with column *amount*, calculate the **total sales amount**.

2 Basic Aggregate – Mean

Using the same *sales* dataset, calculate the **average sale** from column *amount*.

3 Multiple Aggregates Together

Calculate the following from *sales* in a single pipeline:

- Total sales
- Average price
- Maximum price

4 Grouping – Mean by Category

Given a dataset *grades* with columns *student* and *grade*, calculate the **average grade per student**.

5 Multiple Group Aggregates

Using *grades*, calculate for each student:

- Mean grade
- Median grade
- Standard deviation

6 Grouping by Two Variables

Given *sales_data* with columns *location*, *day_of_week*, and *total_sales*, calculate:

- The **average total sales** for each combination of location and day

7 Counting Rows in Groups

Using a dataset *orders* with column *region*, count:

- Number of rows (orders) per region

8 Group + Filter

Given dataset *enrolments* with *course* and *quiz_score*:

- Find courses where the average quiz score is below 70
- Return only those rows

9 Group + Mutate

Using *enrolments*, create a new column:

- Difference between each student's score and their course average

10 Real-World Mini Task

Given *customer_data* with columns:

- *region*, *age*, *purchase_amount*

Perform the following:

- Calculate:
  - Average age
  - Total purchase amount
- Group by *region* and calculate:
  - Average purchase
  - Maximum purchase
- Filter regions where the average purchase is greater than 200

Week 10

Download dataset file from https://github.com/dvn14/Data-Viz-in-R

1 Basic Inner Join

You are given two data frames: *orders* and *customers*

- Task: Perform an inner join using *customer_id*
- Output: Save result as *merged_data*

2 Explore Join Output

Using *merged_data* from Exercise 1:

- Display first 10 rows
- Count number of rows
- Identify if any orders were dropped

3 Left Join

Perform a left join of *orders* with *customers*

- How many rows contain NA values?
- What does this indicate?

4 Full Join

Perform a full join between *orders* and *customers*

- How is this result different from an inner join?

5 Base R Join

Use base R *merge*() to perform:

- Inner join
- Left join ($all.x = TRUE$)

Compare results with *dplyr*

6 Join with Different Column Names

You are given:

- *orders*$*customer_id*
- *customers*$*id*

Perform an inner join using:

```
by = c("customer_id" = "id")
```

7 Rename Before Joining

Task:

- Rename $customers\$id \longrightarrow customer_id$
- Then perform an inner join

Which method is easier to read?

8 Handling Duplicate Column Names

Perform a join where both tables have a column named *date*. Use:

```
suffix = c("_order", "_customer")
```

Why is this important?

9 Three-Table Join

You are given:

- *orders*
- *customers*
- *products*

Join all three tables to create a final dataset with:

- order details
- customer info
- product info

Hint: Chain joins using %>%

10 Business Insight Task

Using the final joined dataset:

- Find total orders per customer
- Identify top 5 customers by number of orders
- Calculate most frequently ordered product

11 Bonus Challenge

Find customers who placed orders but are missing from the customers table
Hint: Use $anti_join()$

Week 11 & 12

Setup:

```
library(ggplot2)
library(dplyr)
```

1 Basic scatter plot with *mtcars*

Create a scatter plot using the *mtcars* dataset

- Put *hp* on the x-axis
- Put *mpg* on the y-axis
- Add points using *geom_point*()
- Give the plot a title: "Horsepower vs MPG"

Goal: practise *ggplot*(), *aes*(), and *geom_point*()

2 Scatter plot with mapped colour using *airquality*

Use the *airquality* dataset to create a scatter plot

- Plot *Ozone* on the x-axis and *Temp* on the y-axis
- Colour the points by *Month*
- Add a smooth trend line

Goal: practise canvas-level aesthetics and geom-level aesthetics, which are both introduced in the lesson

3 Scatter plot with manual aesthetics

Using *airquality*, create the same scatter plot of *Ozone* vs *Temp*, but this time:

- Make all points "darkred"
- Set point size to 3
- Add a title and axis labels

Goal: practise manual aesthetics versus mapped aesthetics, and use *labs*()

4 Bar chart of categories using iris

Create a bar chart using the *iris* dataset

- Put *Species* on the x-axis
- Show the count of each species using *geom_bar*()
- Add a title and axis labels

Goal: practise categorical plots and automatic counts in *geom_bar*(), which is part of the bar-chart content in the slides

5 Histogram using *mtcars*

Create a histogram of fuel efficiency

- Use the *mpg* variable from mtcars
- Add a title: "Distribution of Miles Per Gallon"
- Try changing the number of bins

Goal: practise *geom_histogram*() and interpreting distributions, which is a core Week 12 topic

6 Heatmap / 2D bin plot using *mtcars*

Create a heatmap-style plot using *geom_bin2d*()

- Put *hp* on the x-axis
- Put *wt* on the y-axis
- Use *geom_bin2d*()
- Add a title

Goal: practise frequency visualisation between two continuous variables using *geom_bin2d*()

7 Box plot by category using *mtcars*

Create a box plot that compares fuel efficiency across cylinder groups

- Convert *cyl* to a factor
- Put *cyl* on the x-axis and *mpg* on the y-axis
- Add *geom_boxplot*()
- Add informative labels

Goal: practise box plots for comparing distributions across categories

8 Box plot with fill colour

Using *mtcars*, create another box plot

- Put $as.factor(cyl)$ on the x-axis
- Put *hp* on the y-axis
- Fill the boxes by cylinder group
- Add a title and axis labels

Goal: combine box plots with aesthetic mapping for $fill$, matching the styled box plot example from the slides

9 Summary chart using *iris*

Use *dplyr* to calculate summary statistics for each species

- Find the mean *Petal.Length* for each Species
- Create a bar chart of these mean values
- Use *stat* ="identity"

Goal: practise summarising data first, then plotting the summary, which matches the statistical summary and grouped comparison content

10 Axis customisation using *mtcars*

Create a scatter plot of *hp* vs *mpg*, then customise the axes

- Label the x-axis "Horsepower"
- Label the y-axis "Miles per Gallon"
- Set x-axis limits from 0 to 400
- Set y-axis limits from 0 to 40
- Add a plot title

Goal: practise *scale_x_continuous*() and *scale_y_continuous*() for axis customisation